

FixLink

MIT-WPU Vyas Building Digital Twin & Smart Maintenance Tracker

Comprehensive Project Documentation

Version 1.7.10 | September 2026

Dr. Vishwanath Karad MIT World Peace University, Pune

Document generated: 23 September 2026, 10:00 AM IST

Confidential - MIT-WPU Internal Project

Table of Contents

1. Executive Summary & Project Overview
2. Problem Statement & Solution
3. Target Users & Personas
4. System Architecture & Technology Stack
5. Database Schema & Models
6. Application Modules & Blueprints
7. Core Features - Detailed Breakdown
8. API Endpoints Specification
9. Frontend Architecture & Design System
10. Security & Compliance Protocols
11. Real-Time Communication Layer
12. Smart Scheduling, Ghost Protocol & SLA Engine
13. Testing & Quality Assurance
14. Deployment Architecture
15. Project Rules & Development Guardrails
16. Release History & Changelog
17. File & Directory Structure
18. Setup & Installation Guide
19. Non-Functional Requirements
20. Success Criteria & KPIs

1. Executive Summary & Project Overview

FixLink is a campus facility management system and interactive digital twin designed specifically for the Vyas Building at Dr. Vishwanath Karad MIT World Peace University (MIT-WPU), Pune, India.

The platform bridges the gap between campus occupants (students, faculty, staff) and facility maintenance teams by replacing manual, slow complaint registers with an interactive, SVG-based digital twin map, QR code room detection, instant ticket routing, SLA tracking, and faculty classroom management.

FixLink operates across five distinct portals: a Student/Visitor reporting interface, a Faculty scheduling and room management dashboard, a Professional (Technician) work order portal, an Administrative oversight console, and a Super Admin developer hub for system configuration.

Key Capabilities at a Glance

- **Interactive SVG Digital Twin:** 8-floor vector-rendered building visualization with live room status coloring.
- **QR Code & Direct Reporting:** Instant room detection and issue submission with photo attachments.
- **Automated Ticket Routing:** Categorized dispatch to specialized technicians with SLA timers.
- **Faculty Timetable & Scheduling:** Personal agenda sync, ad-hoc room booking, CSV timetable import, event booking.
- **Real-Time WebSockets:** Pusher-powered instant updates, notification bells, and live map synchronization.
- **Ghost Protocol:** 10-minute check-in enforcement with 3-strike accountability and 7-day suspension.
- **Admin Console:** Unified Faculty Admin Handle for event approvals, classroom management, and ghost protocol monitoring.
- **Dark Mode Support:** Full OLED-optimized dark theme with CSS custom properties throughout.

2. Problem Statement & Solution

2.1 The Problem

- Delayed Maintenance: Broken projectors, faulty ACs, lab equipment failures take days or weeks because reporting relies on physical registers or disconnected emails.
- Ambiguous Location & Asset Details: Technicians receive vague reports ("3rd floor light not working") without knowing the specific room, fixture ID, or severity.
- Zero Visibility & Accountability: Students and faculty receive no feedback on whether their complaint was reviewed, assigned, or when it will be fixed.
- Classroom Clashes & Schedule Blindness: Faculty struggle to find vacant lecture halls for ad-hoc sessions while facility managers lack real-time building occupancy visibility.

2.2 The Solution

- Interactive Vector Digital Twin: Visualizes all 8 floors with SVG layouts that dynamically change colors based on room status (Vacant, Occupied, Issue Reported, In Progress, Fixed).
- Instant QR / Direct Reporting: Users scan a QR code to pre-fill the room number, select issue type, attach a photo, and receive a tracked Ticket Reference.
- Automated Ticket Assignment & Technician Workflow: Issues are categorized and routed to specialized technicians with SLA timers, complexity tiers, and photo proof upon resolution.
- Faculty Timetable & Reservation Hub: Faculty sync teaching schedules, reserve vacant rooms with clash detection, and import master timetables via CSV.
- Real-Time WebSockets & Notifications: Instant multi-screen synchronization via Pusher, in-app toasts, and audio cues.

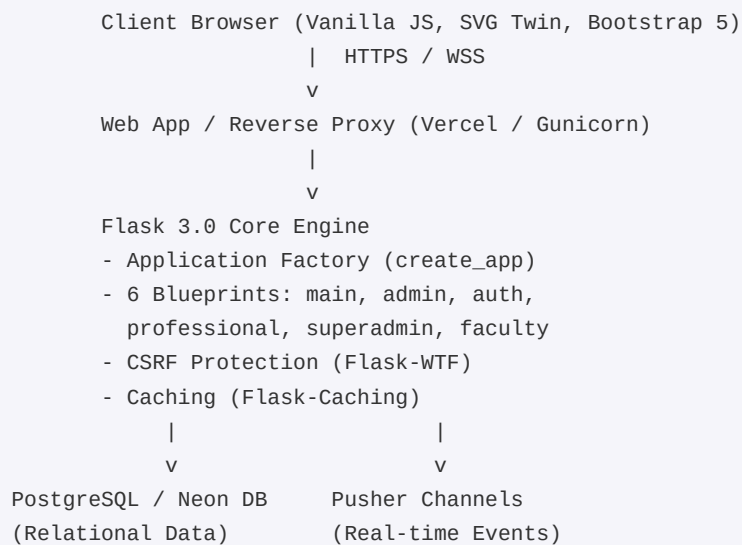
3. Target Users & Personas

Persona	Primary Goal	Key Touchpoints
Students	Report issues quickly	QR Code, Report Page, Ticket Modal
Faculty	Schedule, book rooms	Faculty Portal, Agenda, CSV Sync
Technicians	View & complete work	Professional Dashboard, Task Timer
Admins	Oversee building health	Admin Dashboard, Live Map, Users
Super Admins	System configuration	Developer Hub, DB Migrations

4. System Architecture & Technology Stack

4.1 High-Level Architecture

FixLink is engineered with a Modular Flask Application Factory pattern on the backend, complemented by a Lightweight Vanilla Modern Web Frontend utilizing SVG vector graphics, Bootstrap 5 utilities, and Pusher WebSocket channels for real-time synchronization.



4.2 Backend Technology Stack

- **Language:** Python 3.11+
- **Web Framework:** Flask 3.0.x with Application Factory pattern
- **ORM & Database:** Flask-SQLAlchemy 3.1.x / SQLAlchemy 2.0.x
- **Migrations:** Flask-Migrate 4.0.x (Alembic)
- **Security:** Flask-WTF 1.2.x (CSRF), Werkzeug PBKDF2-SHA256 password hashing
- **Caching:** Flask-Caching 2.1.x
- **Environment:** python-dotenv for .env configuration
- **WSGI Server:** Gunicorn with Eventlet worker (production)

4.3 Database Engine

- **Production:** PostgreSQL (Neon Serverless / Supabase / Render) with NullPool for serverless environments.
- **Development:** SQLite 3 (fixlink.db) with automatic table inspection and initialization.

4.4 Real-Time & Communications

- **WebSocket Engine:** Pusher Channels (pusher>=3.3.0) with pusher-js 8.0.1 client library.
- **Push Notifications:** pywebpush 2.0.x (Web Push API / VAPID protocol).
- **Document Generation:** fpdf2 for audit reports, qrcode for room QR codes, pandas for CSV ingestion.

4.5 Frontend Stack

- **HTML Templates:** Jinja2 template inheritance via base.html with semantic HTML5.
- **CSS:** Custom Design System (style.css) with native CSS Custom Properties (:root and [data-theme="dark"]). Bootstrap 5.3.x loaded locally. Zero Tailwind dependencies.
- **JavaScript:** Native ES Modules (/static/js/modules/*.js) with dynamic imports (api.js, render.js, ui.js, admin_map.js, main.js). Native Touch Event Engine for pinch-to-zoom and drag panning.
- **Icons & Animation:** Bootstrap Icons 1.11.x, GSAP for smooth transitions, SweetAlert2 for dialogs.

4.6 Python Dependencies (requirements.txt)

```
Flask>=3.0.0,<3.2.0
Werkzeug>=3.0.0
Flask-SQLAlchemy>=3.1.0
python-dotenv>=1.0.0
requests>=2.31.0
psycpg2-binary>=2.9.0
Flask-Caching>=2.1.0
Flask-WTF>=1.2.0
qrcode>=7.4.0
fpdf2>=2.7.0
pusher>=3.3.0
cryptography>=42.0.0
pywebpush>=2.0.0
pandas>=2.0.0
Flask-Migrate>=4.0.0
pytz>=2024.1
```

5. Database Schema & Models

FixLink uses 19 SQLAlchemy ORM models spanning building infrastructure, user management, maintenance ticketing, real-time communication, and faculty scheduling. Below is the complete model catalog with key fields and relationships.

User (users)

Unified auth for students, faculty, and admins.

Fields: id, name, prn, email, password_hash, role (student/faculty/admin), is_admin, is_verified, profile_photo, adhoc_suspended_until, created_at

NoShowStrike (no_show_strikes)

Tracks faculty ad-hoc booking no-show violations for 3-strike accountability.

Fields: id, faculty_id (FK->users), booking_id, reason, created_at

SuspensionLog (suspension_logs)

Audit trail for Ghost Protocol suspensions.

Fields: id, faculty_id (FK->users), suspended_at, expires_at, reason, strike_count

Building (buildings)

Campus buildings (Vyas).

Fields: id, name, description

Floor (floors)

Building floors (Ground to 7th).

Fields: id, building_id (FK->buildings), level, name, svg_path, created_at

Room (rooms)

Individual rooms with type and status.

Fields: id, floor_id (FK->floors), number, name, room_type, status, map_coords

Asset (assets)

Equipment within rooms.

Fields: id, room_id (FK->rooms), name, asset_type, status, serial_number

Ticket (tickets)

Maintenance issue reports with full lifecycle tracking.

Fields: id, room_id (FK), asset_id (FK), reporter_id (FK->users), assigned_professional_id (FK), issue_type, description, status, complexity, image_filename, deadline_datetime, job_started_at, job_completed_at

Professional (professionals)

Technicians by category.

Fields: id, name, email, phone, category (Electrician/Plumber/IT/Carpenter/HVAC), is_available, password_hash

HelpRequest (help_requests)

Technician backup requests.

Fields: id, ticket_id (FK), requesting_professional_id (FK), responding_professional_id (FK), status, message

ChatMessage (chat_messages)

Admin-Professional messaging.

Fields: id, sender_type, sender_id, receiver_type, receiver_id, message, is_read, created_at

Notification (notifications)

In-app notification system.

Fields: id, user_id, recipient_role, title, message, type, link, is_read, created_at

PushSubscription (push_subscriptions)

Web Push subscription endpoints.

Fields: id, user_id, endpoint, p256dh, auth, created_at

AdHocBooking (adhoc_bookings)

Instant room reservations with check-in validation.

Fields: id, faculty_id (FK->users), room_id (FK), subject, start_datetime, end_datetime, checked_in, status

Timetable (timetable_entries)

Master academic schedule entries.

Fields: id, room_id (FK), faculty_id (FK), day_of_week, start_time, end_time, subject, course, year, division

RoomBooking (room_bookings)

Formal classroom reservations.

Fields: id, faculty_id (FK), room_id (FK), subject, slot_start, slot_end, status, source, created_at

BugReport (bug_reports)

User-submitted application bug reports.

Fields: id, reporter_name, reporter_email, description, severity, status, created_at

ScheduleSubmission (schedule_submissions)

Faculty CSV timetable upload tracking.

Fields: id, faculty_id (FK), filename, record_count, status, created_at

EventBooking (event_bookings)

Multi-room/floor event reservations.

Fields: id, title, description, faculty_id (FK), booking_type (rooms/floors), start_date, end_date, start_time, end_time, status, rejection_reason, created_at

6. Application Modules & Blueprints

FixLink follows the Flask Blueprint pattern to organize routes into six independent modules, each with its own routes.py and template directory.

- **main (/report, /):** Student/visitor portal: landing page, issue reporting with interactive SVG map, QR code auto-detection, ticket submission, room and asset APIs.
- **auth (/login, /signup, /logout):** Unified authentication: login for students, faculty, admin, professional; signup with MIT-WPU email verification; password reset; case-insensitive email matching.
- **admin (/admin/*):** Administrative dashboard: ticket management, technician dispatch, live status map, user management, analytics, Faculty Admin Handle (events, CMM, ghost protocol, booking history), professional management, chat.
- **faculty (/faculty/*):** Faculty portal: personal dashboard with weekly agenda, ad-hoc room booking, timetable CSV import, event booking (/events/book), event cancellation, real-time room tracker with Digital Twin.
- **professional (/professional/*):** Technician portal: task queue, job start/stop timers, complexity assessment, help requests, photo proof upload, SLA countdown.
- **superadmin (/superadmin/*):** Developer hub: system diagnostics, database migrations, global user role management, telemetry, audit logs.

Backend Service Modules

- **app/__init__.py:** Application Factory (create_app), core SQLAlchemy & CSRF initialization, session lockdown configuration, blueprint registration, database auto-initialization.
- **app/models.py:** 19 SQLAlchemy ORM models (1,422 lines) covering the complete data schema.
- **app/analytics.py:** Administrative analytics engine: MTTR calculation, SLA compliance rates, asset failure rankings, volume trend analysis.
- **app/api_utils.py:** Standardized JSON API response helpers (api_response function).
- **app/cache.py:** Flask-Caching configuration and floor plan query memoization.
- **app/database.py:** Database initialization, auto-seeding of buildings/floors/rooms/assets, credential auto-repair.
- **app/decorators.py:** Authentication decorators: @admin_required, @faculty_login_required, @professional_required, @handle_api_errors, rate limiting.
- **app/realtime.py:** Pusher WebSocket event dispatching: room status updates, task notifications, admin alerts.
- **app/scheduler.py:** Background scheduler: unassigned ticket alerts (>24h), overdue SLA alerts, Ghost Protocol no-show cleanup.
- **app/sla_service.py:** SLA deadline calculation, complexity-based timer assignment, overdue escalation logic.
- **app/utils.py:** Shared utilities: email sending (EmailJS), web push notifications, file upload validation, QR code generation.

7. Core Features - Detailed Breakdown

7.1 Interactive SVG Digital Twin

The heart of FixLink is an interactive vector floor plan system that provides a live "digital twin" of the Vyas Building.

- Multi-Floor Vector Layouts: SVG-rendered architectural plans for Ground Floor through 7th Floor (8 total).
- Live Room State Coloring: Normal (Green), Issue Reported (Red), In Progress (Amber), Assigned (Blue), Vacant (Neutral).
- Interactive Gestures: Two-finger mobile pinch-to-zoom (0.6x to 4.5x), drag pan, double-tap zoom toggle, desktop scroll zoom.
- Deep-Linking: URL parameters (?floor=4&room=VY401) auto-select, zoom, and highlight target rooms with pulsating glow.
- Classroom Flashlight Mode: Click a class card to switch to the map tab, auto-load the correct floor SVG, grayscale all other rooms, and spotlight-pulse the target room in green.

7.2 Maintenance Issue Reporting

- Room Auto-Detection: Automatically detects room from QR URL (/report?room=VY402).
- Validated Input Controls: Student PRN verification, MIT-WPU email restriction (@mitwpu.edu.in), camera photo attachment.
- Reference Tracking: Unique integer Ticket IDs (#1042) with one-click clipboard copy.
- Issue Categories: Electrical, Plumbing, IT/AV, Carpentry, HVAC, General, Hygiene.
- Image Upload: Up to 16MB with secure_filename sanitization; supports PNG, JPG, JPEG, WEBP.

7.3 Technician Work Order System

- Specialization Matching: Electrician, Plumber, IT/AV Tech, Carpenter, HVAC categories.
- Job Lifecycle: Accept Task > Start Timer > Set Complexity (Low/Medium/High) > Complete with Photo Proof or Cancel with Reason.
- SLA Countdown Timers: Complexity-based deadlines with automated overdue escalation to admin.
- Collaborative Help Requests: Technicians dispatch backup requests to other available technicians.

7.4 Faculty Portal & Room Scheduling

- Weekly Agenda View: Chronological lecture, practical, and ad-hoc session display with "Today" highlighting.
- Ad-Hoc Room Reservation: Select floor/room/time with automated clash prevention against master timetables. 15-minute transition dead-zone buffer. 10-minute Ghost Protocol check-in requirement.
- CSV Master Timetable Import: Client-side dry-run parsing, error detection, and database commit for academic schedule loading.
- Event Booking System: Multi-room and entire-floor reservation requests with admin approval pipeline. Faculty can cancel pending/approved events anytime.
- Digital Twin Integration: Real-time room occupancy tracker embedded in faculty dashboard.

7.5 Administrative Console

- Live Floor Status Map: Interactive digital twin with slide-out room detail drawers and one-click technician dispatch.
- Faculty Admin Handle: Unified 4-tab console consolidating Event Approvals, Classroom Management (CMM), Ghost Protocol monitoring, and Booking History.
- Event Approvals: Pending/approved/rejected/cancelled event management with conflict detection and automatic lecture displacement notifications.
- User Management: Add, edit, verify, promote, or suspend user accounts across Student, Faculty, and Admin tiers.
- Analytics Dashboard: Mean Time to Resolution (MTTR), SLA compliance rates, top failing assets, and volume trend charts.
- Professional Management: Technician roster, availability toggles, assignment history, performance metrics.
- Admin-Professional Chat: Real-time messaging system for coordination during complex maintenance operations.

7.6 Notification & Alerting System

- In-App Notification Bell: Global header bell icon with unread badge counter and modal history list.
- Real-Time Pusher Events: Channel-based notifications for admin-notifications, faculty-alerts, room-status-updates, professional-tasks.
- Web Push Notifications: VAPID-based browser push for offline alerts.
- SweetAlert2 Interactive Dialogs: Confirmation modals for destructive actions (event cancellation, ticket rejection, timetable deletion).

8. API Endpoints Specification

8.1 Digital Twin & Map Data

Method	Endpoint	Access	Description
GET	/api/floors/<building_id>	Auth	Floors list for building
GET	/api/rooms/floor/<floor_id>	Auth	Room list with status metrics
GET	/api/room/<room_number>	Auth	Room metadata & active issues
GET	/admin/floor-data/<floor_id>	Admin	Full room/asset/ticket payload

8.2 Maintenance & Tickets

Method	Endpoint	Access	Description
POST	/report	Public	Create maintenance ticket
GET	/admin/api/ticket/<id>	Admin	Ticket details & timestamps
POST	/admin/api/ticket/<id>/assign	Admin	Assign professional
POST	/api/task/<id>/start	Pro	Start work timer
POST	/api/task/<id>/complete	Pro	Submit completion proof

8.3 Faculty Scheduling & Events

Method	Endpoint	Access	Description
GET	/faculty/api/occupancy	Faculty	Real-time room occupancy
POST	/faculty/api/claim-room	Faculty	Ad-hoc room reservation
POST	/faculty/events/book	Faculty	Submit event booking
POST	/faculty/events/<id>/cancel	Faculty	Cancel own event
POST	/admin/events/<id>/approve	Admin	Approve event booking
POST	/admin/events/<id>/reject	Admin	Reject/revoke event

9. Frontend Architecture & Design System

9.1 Design Philosophy

FixLink adheres to a "Clean Academic High-Tech Digital Twin" design language, combining the prestige of the MIT-WPU brand with responsive operational cockpit aesthetics: glassmorphic surfaces, crisp contrast ratios, and responsive vector floor maps.

9.2 Color Palette & Brand Tokens

Token	Hex	Usage
--mitwpu-blue	#0B4D8C	Primary brand, headers, actions
--mitwpu-blue-dark	#083968	Hover states, active sidebar
--mitwpu-red	#C8102E	University crimson, warnings
--lab-teal	#20C997	Lab rooms, digital twin markers
Normal/Fixed/Vacant	#28A745	Operational, resolved, vacant
Issue Reported	#DC3545	Unassigned issues, broken assets
In Progress	#FFC107	Technician on site
Assigned/Booked	#0D6EFD	Assigned, lecture in progress

9.3 JavaScript Module Architecture

- **modules/render.js:** SVG floor plan rendering, room polygon generation, Flashlight spotlight mode, dynamic floor loading.
- **modules/api.js:** Centralized fetch wrapper for all REST API calls with CSRF token injection.
- **modules/ui.js:** UI state management, theme toggling, sidebar controls, responsive layout adjustments.
- **modules/admin_map.js:** Admin-specific status map interactions, slide-out drawer controls, technician dispatch.
- **modules/main.js:** Application bootstrap, event listener initialization, touch gesture engine.
- **notifications.js:** Pusher channel subscriptions, notification bell badge, toast alerts, modal history.
- **animations.js:** GSAP entry animations, card stagger effects, page transition controllers.

10. Security & Compliance Protocols

- CSRF Enforcement: All mutable requests (POST, PUT, DELETE) require a valid CSRF token via `csrf_token()` or `X-CSRFToken` header. Enforced globally by Flask-WTF.
- Session Security: Cookies configured with `HttpOnly=True`, `SameSite=Lax`, `Secure=True`. 2-hour idle session expiration (`PERMANENT_SESSION_LIFETIME`).
- Password Encryption: PBKDF2 with SHA-256 via Werkzeug `generate_password_hash` / `check_password_hash`. No plaintext storage.
- SQL Injection Protection: All queries use SQLAlchemy ORM parameter binding. No raw SQL string concatenation.
- File Upload Hardening: 16MB max upload size, `secure_filename()` sanitization, strict image extension whitelist (PNG, JPG, JPEG, WEBP).
- Case-Insensitive Auth: All email lookups use `func.lower()` to prevent case-sensitivity login failures on PostgreSQL.
- Role-Based Access Control: `@admin_required`, `@faculty_login_required`, `@professional_required` decorators on all protected routes.
- Resource Ownership: State-mutating API routes verify `event.faculty_id == session.user_id` before allowing cancellation/modification.
- Rate Limiting: Sensitive endpoints (login, report) protected against brute-force and bot spamming.
- Credential Auto-Seeding: Core admin/dev accounts auto-created on cold boot with self-healing password repair.
- Database Initialization Safety: Table existence checks via `sqlalchemy.inspect` before queries during cold starts.

11. Real-Time Communication Layer

FixLink uses Pusher Channels for WebSocket-based real-time communication across all portals.

Pusher Event Channels

- **admin-notifications:** New ticket alerts, event requests, event cancellation notices, technician status changes.
- **room-status-updates:** Live room color changes on the Digital Twin map when tickets are filed, assigned, or resolved.
- **professional-tasks:** New task assignments, help request dispatches to individual technicians.
- **faculty-{id}-alerts:** Personal faculty notifications for event approvals, rejections, lecture displacements.
- **live-map:** Refresh grid events triggered when events are approved, cancelled, or revoked to update building occupancy.

12. Smart Scheduling, Ghost Protocol & SLA Engine

12.1 Ghost Protocol (No-Show Enforcement)

- 10-Minute Check-In Timeout: All ad-hoc bookings require explicit check-in within 10 minutes of the scheduled start time.
- Automatic Cancellation: Unconfirmed bookings are auto-cancelled by the background scheduler, immediately releasing room occupancy.
- 3-Strike Accountability: No-show violations tracked in a 30-day rolling window (NoShowStrike model). Three strikes trigger automatic 7-day ad-hoc booking suspension.
- Suspension Audit Trail: SuspensionLog model records all suspensions with timestamps, expiry dates, and strike counts.
- Admin Override: Administrators can lift suspensions and clear strikes via the Ghost Protocol tab in Faculty Admin Handle.

12.2 Physical Transition Buffers

All room availability calculations inject a 15-minute transition dead-zone after every occupied session to prevent physical hallway bottleneck clashes between consecutive classes.

12.3 SLA Engine

- Complexity-Based Deadlines: Low (4h), Medium (8h), High (24h) SLA targets based on technician-assessed complexity.
- Overdue Escalation: Background scheduler alerts administrators when tickets breach their SLA deadline.
- MTTR Tracking: Mean Time to Resolution calculated and displayed on the admin analytics dashboard.

13. Testing & Quality Assurance

FixLink maintains a comprehensive automated test suite using pytest with 51 tests across 10 test modules. All tests run against an in-memory SQLite database with mocked external services.

Test Suite Overview

Test Module	Tests	Coverage Area
test_analytics.py	3	Admin analytics engine, MTTR calc
test_auth.py	8	Login, signup, session, password
test_bulk_assign.py	2	Bulk technician assignment
test_csv_import.py	7	Timetable CSV dry-run & commit
test_event_cancellation.py	3	Faculty cancel, admin revoke
test_rule7_verification.py	8	Pre-commit verification checks
test_super_admin_guard.py	8	Super admin access control
test_ticket_lifecycle.py	1	Full ticket open-to-close cycle
test_timetable_sanitizer.py	11	Timetable data validation
TOTAL	51	100% pass rate, 0 regressions

Testing infrastructure: conftest.py provides shared fixtures (app, client, admin_user, student_user, professional_user) with in-memory SQLite and mocked external services (EmailJS, WebPush).

14. Deployment Architecture

14.1 Docker Deployment

FixLink ships with a production-ready Dockerfile based on Python 3.11-slim with Gunicorn and Eventlet workers.

- Base Image: python:3.11-slim with gcc, libffi-dev, libpq-dev for cryptography and PostgreSQL support.
- WSGI Server: Gunicorn with eventlet worker class for WebSocket compatibility.
- Auto-Migration: Container runs "flask db upgrade" before starting the server.
- Docker Compose: docker-compose.yml provided for single-command deployment.

14.2 Vercel Serverless Deployment

- vercel.json configuration for serverless Python function deployment.
- ProxyFix middleware for Vercel reverse proxy header support.
- NullPool connection pooling to prevent PostgreSQL connection exhaustion.
- Environment variables managed through Vercel dashboard.

14.3 Environment Variables

Variable	Default	Description
SECRET_KEY	Auto-generated	Flask session encryption key
DATABASE_URL	postgresql://...	PostgreSQL connection string
PORT	5000	Server port
FLASK_DEBUG	True	Debug mode toggle
PUSHER_APP_ID	Required	Pusher app identifier
PUSHER_KEY	Required	Pusher public key
PUSHER_SECRET	Required	Pusher secret key
VAPID_PRIVATE_KEY	Required	Web Push VAPID private key

15. Project Rules & Development Guardrails

FixLink enforces strict development protocols documented in Rules.md. All contributors must adhere to these guardrails before committing changes.

- **Rule 1 - Performance:** Lighthouse 95+, FCP <1.5s, LCP <2.5s. No raster images for floor plans. Only inline SVGs. No unnecessary dependencies.
- **Rule 2 - SVG Architecture:** No raw Figma exports. Strip all inline fill/stroke attributes. Use .interactive-room classes. Maintain pointer-events: none on containers.
- **Rule 3 - Dark Mode Integrity:** No hardcoded colors. All colors must use CSS variables. SVG rooms use low-opacity fills with solid strokes.
- **Rule 4 - Database Safety:** Check table existence before startup queries. Auto-seed on cold boot. Provide migration logic for schema changes.
- **Rule 5 - Boy Scout Rule:** Zero-debt policy. Scan surrounding 50 lines when fixing bugs. Aggressively delete dead code, unused imports, and duplicate logic.
- **Rule 6 - Auth & Credentials:** Never reset existing credentials during migrations. Auto-seed core accounts. Case-insensitive email matching. Self-healing password repair.
- **Rule 7 - Pre-Commit Verification:** 15-point verification checklist covering credentials, dark mode, mobile responsiveness, OWASP security, CSRF, IDOR, Lighthouse, and automated pytest execution.
- **Rule 8 - Scheduling & Ghost Protocol:** 15-minute transition dead-zones, 10-minute check-in validation, 3-strike 7-day suspension lockout enforcement.

16. Release History & Changelog

Version	Date	Focus Areas
v1.7.10	2026-09-22	Faculty Event Cancellation & Admin Event Revocation
v1.7.9	2026-09-22	Unified Faculty Admin Handle, UI Color Fix, Booking History
v1.7.8	2026-09-22	Event Booking, Multi-Room Allocation, Notifications
v1.7.7	2026-09-22	Classroom Flashlight, Ad-Hoc Highlighting, Cleanup
v1.7.6	2026-09-16	Admin CMM UX Redesign & Safety Protection
v1.7.5	2026-09-16	Faculty Portal Core + 3-Strike Ghost Protocol
v1.7.4	2026-09-15	CSV Timetable Import Pipeline
v1.7.3	2026-09-14	Faculty Dashboard Full Build
v1.7.2	2026-09-10	Ghost Protocol Background Scheduler
v1.7.1	2026-09-08	Ad-Hoc Room Booking System
v1.7.0	2026-09-05	Faculty Portal Foundation

17. File & Directory Structure

```

FixLink/
|-- run.py                # Entry point
|-- requirements.txt      # Python dependencies
|-- Dockerfile           # Docker production config
|-- docker-compose.yml   # Docker Compose
|-- vercel.json           # Vercel serverless config
|-- pytest.ini           # Test configuration
|-- .env                 # Environment variables
|-- PRD.md, TechSpec.md, # Product & Tech docs
|   AppFlow.md, Design.md # UX & Design docs
|-- Rules.md             # Development guardrails
|-- Tracker.md           # Release changelog
|
|-- app/
|   |-- __init__.py       # App Factory
|   |-- models.py         # 19 ORM Models (1,422 lines)
|   |-- analytics.py      # Analytics engine
|   |-- api_utils.py      # JSON response helpers
|   |-- cache.py          # Caching config
|   |-- database.py       # DB init & seeding
|   |-- decorators.py     # Auth decorators
|   |-- realtime.py       # Pusher WebSocket events
|   |-- scheduler.py      # Background scheduler
|   |-- sla_service.py    # SLA engine
|   |-- utils.py          # Shared utilities
|   |
|   |-- blueprints/
|   |   |-- admin/        # Admin routes
|   |   |-- auth/         # Authentication routes
|   |   |-- faculty/      # Faculty routes
|   |   |-- main/         # Student/Visitor routes
|   |   |-- professional/ # Technician routes
|   |   |-- superadmin/   # Developer routes
|   |
|   |-- templates/        # 35+ Jinja2 templates
|   |-- static/
|   |   |-- css/style.css # Design system
|   |   |-- js/modules/   # ES Module architecture
|   |   |-- images/, img/ # Static assets
|   |
|-- tests/                # 10 test modules, 51 tests
|-- scripts/              # Init data, QR generation
|-- migrations/           # Alembic migrations
|-- svg_maps/             # Source SVG floor plans
|-- qr_codes/             # Generated room QR codes

```

18. Setup & Installation Guide

Prerequisites

- Python 3.11 or higher
- PostgreSQL database (or SQLite for local development)
- Pusher account for real-time features
- Git for version control

Installation Steps

1. Clone the repository:

```
git clone https://github.com/meowpher/FixLink.git
cd FixLink
```

2. Create and activate virtual environment:

```
python -m venv venv
venv\Scripts\activate (Windows)
source venv/bin/activate (macOS/Linux)
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Configure environment:

Copy `.env.example` to `.env` and fill in credentials
(`DATABASE_URL`, `SECRET_KEY`, `PUSHER_*`, `VAPID_*`)

5. Initialize database:

```
python scripts/init_data.py
```

6. Run the application:

```
python run.py
Server starts at http://localhost:5000
```

7. Run tests:

```
py -m pytest (expects 51/51 passing)
```

Application URLs

Endpoint	Description
/	Landing page with floor guide
/report	Student issue reporting portal

/report?room=VY404	Auto-select room via QR
/login	Unified login page
/admin	Admin dashboard
/admin/status-map	Live digital twin status map
/faculty/dashboard	Faculty portal
/professional/dashboard	Technician work orders
/superadmin/dashboard	Developer hub

19. Non-Functional Requirements

- Performance: Lighthouse Score ≥ 95 ; FCP $\leq 1.5s$; LCP $\leq 2.5s$.
- Mobile-First Responsiveness: Fully responsive down to 360px width. Optimized for iOS Safari and Android Chrome.
- Theme Adaptability: Complete dual-theme support (Light + OLED Dark Mode #0F1117) with no un-styled elements.
- Accessibility: Semantic HTML5 structure, descriptive aria-labels on all interactive elements.
- Security: CSRF on all forms, HttpOnly cookies, PBKDF2 password hashing, SQLAlchemy parameterized queries.
- Reliability: Database initialization safety with table existence checks. Auto-recovery on cold starts.
- Scalability: Blueprint modular architecture. NullPool for serverless. Flask-Caching for query memoization.

20. Success Criteria & KPIs

- Report Time: ≤ 30 seconds for a student to report an issue via QR code.
- Assignment Velocity: ≤ 10 minutes median time from ticket submission to technician assignment.
- Mean Time to Resolution: Decreased by 45% compared to paper-based maintenance registers.
- Faculty Scheduling Friction: Zero double-booking conflicts across lecture halls.
- System Uptime: 99.9% during university operating hours (07:00 - 21:00 IST).
- Test Suite: 100% pass rate with zero regressions across all 51 automated tests.

End of Documentation

FixLink v1.7.10 | MIT-WPU Internal Project
Dr. Vishwanath Karad MIT World Peace University, Pune

Generated: 23 September 2026, 10:00 AM IST
This document is confidential and intended for internal use only.